LAB EXPERIMENTS

Using Unit 3 Concepts

Lab Exercise 8: Contact Book

Lab Exercise 9: Student Grading System

Lab Exercise 10: Frequency Counter

Lists and dictionaries: complete programs, test cases, verified outputs and real-world applications.

NAME	Sriram Satvik Karanam
USN	25BTRAD049
BRANCH	CSE - AIDE (B Section)
COURSE	Python Programming

Overview

This report solves Lab Exercises 8, 9 and 10 of Python Programming (Unit 3). Each exercise starts with its **highlighted problem statement** exactly as given, followed by requirement analysis, data-structure selection, the complete source code, test cases with real output, and a real-world application.

Exercises at a glance

Lab	Program	Main data structures	Key Python concepts
8	Contact Book	List of dictionaries	CRUD operations, validation, searching, sorting, exception handling
9	Student Grading System	Dictionary of dictionaries + list	if / elif / else, grade boundaries, ranking, statistics
10	Frequency Counter	Dictionary {item: count}	Counting loop, dict.get(), sorting with lambda, string cleaning

What CT and RWE mean in this report

CT	**Critical Thinking:** analysing the requirements, choosing suitable data structures, examining boundary values and verifying that the output is correct.
RWE	**Real World Exposure:** applying each program to a realistic scenario (club or fest contacts, a class result sheet, survey and feedback data) and interpreting the findings.

Files delivered

- Programs: `contact_book.py`, `grading_system.py`, `frequency_counter.py` (each has a menu-driven interface).
- Tests: `test_contact_book.py`, `test_grading.py`, `test_frequency.py`.
- Every output printed in this report was produced by actually running these test files. Python 3.12, standard library only.

LAB EXERCISE 8	**Contact book using lists and dictionaries.** Contact Book Using Lists and Dictionaries: Develop a Python program to add, search, update, delete, and display contact details using lists and dictionaries.
CT	Analyze requirements, select suitable data structures, implement operations, and test correctness. (CT)
RWE	Apply the contact book to a personal or organizational contact management scenario. (RWE)

8.1 Aim

To build a menu-driven contact book that can add, search, update, delete and display contacts, store each contact as a dictionary inside a list, and then use it for a personal address book and for an organisation (a fest organising committee).

8.2 Requirement analysis (CT)

Requirement	Analysis	Implemented as
Add	Each contact needs name, phone, email, category. Bad data must be rejected: empty name, invalid phone or email, duplicate phone.	add_contact()
Search	Users remember different things (part of a name, a phone, a group), so search must work on any field and ignore case.	search(query, field)
Update	Any field can change. The contact must be located reliably, even if two people share a name.	update_contact(phone, field, value)
Delete	Remove exactly one contact and give a clear error if it does not exist.	delete_contact(phone)
Display	Tidy, numbered, sorted table with a count; friendly message when nothing matches.	display(contacts, sort_by)
Correctness	Behaviour must be proven with normal, boundary and error cases.	7 tests + RWE scenario + assertions

8.3 Selecting the data structure (CT)

Option	Verdict	Reason
Parallel lists (names[], phones[] ...)	Rejected	Easily go out of sync; delete or update must touch every list.
Dictionary keyed by name	Rejected	Two people with the same name would overwrite each other.
Dictionary keyed by phone	Possible	Fast lookup, but sorting and searching by other fields still needs scanning.
List of dictionaries	**Chosen**	One self-contained record per contact; new fields are easy to add; list comprehensions search any field; sorted() orders by any field; duplicate names are allowed.

Trade-off: finding a contact by phone scans the list (linear time). This is perfectly fine for hundreds of contacts; for thousands, a dictionary index {phone: record} could be added alongside the list. The phone number is used

as the unique key and is checked in code.

8.4 Methods

Method	Purpose
clean_phone / valid_phone / valid_email	Normalise and validate input (10 digits starting 6-9, simple email pattern)
add_contact(name, phone, email, category)	Validate, normalise (Title Case, lower-case email) and append a new dictionary
find_by_phone(phone)	Return the matching contact or None
search(query, field)	Case-insensitive partial match on name, phone, email; exact match on category
update_contact(phone, field, new_value)	Validate and change one field; returns the old value
delete_contact(phone)	Remove the contact; raises KeyError if missing
display(contacts, sort_by) / group_by_category()	Print a sorted table; build {category: [names]} using setdefault()

8.5 Source code: contact_book.py

```
"""
Lab Exercise 8 : Contact Book using Lists and Dictionaries
Author : Sriram Satvik Karanam | USN : 25BTRAD049 | CSE-AIDE B

Data structure used
-------------------
contacts : list of dict
    [{"name": "Asha Rao", "phone": "9876543210",
      "email": "asha@example.com", "category": "Friend"}, ...]

The phone number is treated as the unique key of a contact because two
different people can share the same name but never the same phone number.
"""
import re

FIELDS = ("name", "phone", "email", "category")


class ContactBook:
    def __init__(self):
        self.contacts = []          # list of dictionaries

    # ------------------------------------------------------------------
    # Validation helpers
    # ------------------------------------------------------------------
    @staticmethod
    def clean_phone(phone):
        return re.sub(r"[\s\-+]", "", phone.strip())

    @staticmethod
    def valid_phone(phone):
        """10 digits, first digit between 6 and 9 (Indian mobile numbers)."""
        return bool(re.fullmatch(r"[6-9]\d{9}", phone))

    @staticmethod
    def valid_email(email):
        """Email is optional; if given it must look like name@domain.ext"""
        return email == "" or bool(re.fullmatch(r"[\w.+-]+@[\w-]+(\.[\w-]+)+", email))

    # ------------------------------------------------------------------
    # ADD
    # ------------------------------------------------------------------
    def add_contact(self, name, phone, email="", category="Other"):
        name = name.strip().title()
        phone = self.clean_phone(phone)
```

```
        email = email.strip().lower()
        category = category.strip().title() or "Other"
        if not name:
            raise ValueError("Name cannot be empty.")
        if not self.valid_phone(phone):
            raise ValueError(f"Invalid phone number '{phone}' (need 10 digits starting 6-9).")
        if not self.valid_email(email):
            raise ValueError(f"Invalid email '{email}'.")
        if self.find_by_phone(phone):
            raise ValueError(f"A contact with phone {phone} already exists.")
        contact = {"name": name, "phone": phone, "email": email, "category": category}
        self.contacts.append(contact)
        return contact

    # ------------------------------------------------------------------
    # SEARCH
    # ------------------------------------------------------------------
    def find_by_phone(self, phone):
        phone = self.clean_phone(phone)
        for contact in self.contacts:
            if contact["phone"] == phone:
                return contact
        return None

    def search(self, query, field="name"):
        """Case-insensitive search. Returns a list of matching contacts."""
        if field not in FIELDS:
            raise ValueError(f"Search field must be one of {FIELDS}.")
        query = query.strip().lower()
        if field == "category":
            return [c for c in self.contacts if c["category"].lower() == query]
        return [c for c in self.contacts if query in c[field].lower()]

    # ------------------------------------------------------------------
    # UPDATE
    # ------------------------------------------------------------------
    def update_contact(self, phone, field, new_value):
        contact = self.find_by_phone(phone)
        if contact is None:
            raise KeyError(f"No contact found with phone {phone}.")
        if field not in FIELDS:
            raise ValueError(f"Field must be one of {FIELDS}.")
        new_value = new_value.strip()
        if field == "name":
            new_value = new_value.title()
            if not new_value:
                raise ValueError("Name cannot be empty.")
        elif field == "phone":
            new_value = self.clean_phone(new_value)
            if not self.valid_phone(new_value):
                raise ValueError(f"Invalid phone number '{new_value}'.")
            if self.find_by_phone(new_value):
                raise ValueError(f"Phone {new_value} already belongs to another contact.")
        elif field == "email":
            new_value = new_value.lower()
            if not self.valid_email(new_value):
                raise ValueError(f"Invalid email '{new_value}'.")
        else:
            new_value = new_value.title() or "Other"
        old_value = contact[field]
        contact[field] = new_value
        return old_value

    # ------------------------------------------------------------------
    # DELETE
    # ------------------------------------------------------------------
    def delete_contact(self, phone):
        contact = self.find_by_phone(phone)
        if contact is None:
            raise KeyError(f"No contact found with phone {phone}.")
        self.contacts.remove(contact)
        return contact

    # ------------------------------------------------------------------
    # DISPLAY
    # ------------------------------------------------------------------
```

```
    def display(self, contacts=None, sort_by="name"):
        rows = self.contacts if contacts is None else contacts
        if not rows:
            print("No contacts to display.")
            return
        rows = sorted(rows, key=lambda c: (c[sort_by].lower(), c["name"].lower()))
        head = f"{'No.':<5}{'Name':<20}{'Phone':<13}{'Category':<12}Email"
        print(head)
        print("-" * 70)
        for i, c in enumerate(rows, start=1):
            print(f"{i:<5}{c['name']:<20}{c['phone']:<13}{c['category']:<12}{c['email'] or '-'}")
        print(f"\nTotal: {len(rows)} contact(s)")

    def group_by_category(self):
        groups = {}
        for c in self.contacts:
            groups.setdefault(c["category"], []).append(c["name"])
        return groups


# ---------------------------------------------------------------------
# Menu driven program
# ---------------------------------------------------------------------
def menu():
    book = ContactBook()
    options = """
===== CONTACT BOOK =====
1. Add contact        4. Delete contact
2. Search contact     5. Display all
3. Update contact     6. Exit
"""
    while True:
        print(options)
        choice = input("Enter choice: ").strip()
        try:
            if choice == "1":
                c = book.add_contact(input("Name: "), input("Phone: "),
                                     input("Email (optional): "), input("Category: "))
                print("Added:", c["name"])
            elif choice == "2":
                field = input("Search by (name/phone/email/category): ").strip().lower()
                book.display(book.search(input("Search text: "), field))
            elif choice == "3":
                old = book.update_contact(input("Phone of contact: "),
                                          input("Field to change: ").strip().lower(),
                                          input("New value: "))
                print("Updated. Old value was:", old)
            elif choice == "4":
                print("Deleted:", book.delete_contact(input("Phone of contact: "))["name"])
            elif choice == "5":
                book.display()
            elif choice == "6":
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")
        except (ValueError, KeyError) as err:
            print("Error:", err)


if __name__ == "__main__":
    menu()
```

8.6 Test program: test_contact_book.py

```python
from contact_book import ContactBook


def banner(title):
    print("=" * 66)
    print(title)
    print("=" * 66)


# ---------------- Test 1: ADD ----------------
banner("TEST 1: Adding personal contacts")
book = ContactBook()
data = [
    ("asha rao",     "98765 43210", "asha.rao@gmail.com",   "friend"),
    ("Ravi Kumar",   "9123456780",  "ravi.k@outlook.com",   "family"),
    ("Meera Nair",   "8899776655",  "meera.nair@gmail.com", "family"),
    ("Dr. Suresh",   "7012345678",  "suresh@college.edu",   "work"),
    ("Karthik S",    "9000011111",  "",                     "friend"),
    ("Asha Verma",   "6001122334",  "asha.v@yahoo.com",     "work"),
]
for d in data:
    book.add_contact(*d)
book.display()
assert len(book.contacts) == 6

# ---------------- Test 2: SEARCH ----------------
banner("TEST 2: Searching")
print("Search name 'asha' (partial, case-insensitive):")
book.display(book.search("ASHA"))
print("\nSearch phone '9123':")
book.display(book.search("9123", "phone"))
print("\nSearch category 'family':")
book.display(book.search("family", "category"))
print("\nSearch name 'zzz' (no match):")
book.display(book.search("zzz"))
assert len(book.search("asha")) == 2
assert len(book.search("family", "category")) == 2
assert book.search("zzz") == []

# ---------------- Test 3: UPDATE ----------------
banner("TEST 3: Updating")
print("Email changed from:", book.update_contact("9000011111", "email", "karthik.s@gmail.com") or "(empty)")
print("Category changed from:", book.update_contact("9000011111", "category", "colleague"))
print("Phone changed from:", book.update_contact("9123456780", "phone", "9123400000"))
book.display(book.search("karthik"))
book.display(book.search("ravi"))
assert book.find_by_phone("9123456780") is None
assert book.find_by_phone("9123400000")["name"] == "Ravi Kumar"

# ---------------- Test 4: DELETE ----------------
banner("TEST 4: Deleting")
removed = book.delete_contact("6001122334")
print("Deleted:", removed["name"], "|", removed["phone"])
print("Contacts left:", len(book.contacts))
print("Search 'verma' now ->", book.search("verma") or "no match (deletion verified)")
try:
    book.delete_contact("6001122334")
except KeyError as e:
    print("Deleting again   -> caught:", e)
assert len(book.contacts) == 5

# ---------------- Test 5: VALIDATION ----------------
banner("TEST 5: Validation and error handling")
cases = [
    ("Empty name",        lambda: book.add_contact("  ", "9876500000")),
    ("Short phone",       lambda: book.add_contact("Test", "98765")),
    ("Starts with 5",     lambda: book.add_contact("Test", "5876543210")),
    ("Letters in phone",  lambda: book.add_contact("Test", "98ab543210")),
    ("Bad email",         lambda: book.add_contact("Test", "9876500001", "not-an-email")),
    ("Duplicate phone",   lambda: book.add_contact("Clone", "9876543210")),
    ("Update unknown",    lambda: book.update_contact("9999999999", "email", "a@b.com")),
    ("Bad update field",  lambda: book.update_contact("9876543210", "age", "20")),
    ("Phone clash",       lambda: book.update_contact("9876543210", "phone", "8899776655")),
    ("Bad search field",  lambda: book.search("x", "address")),
```

```
]
for label, action in cases:
    try:
        action()
        print(f"{label:<18}: NO ERROR (unexpected)")
    except (ValueError, KeyError) as e:
        print(f"{label:<18}: caught -> {e}")

# ---------------- Test 6: GROUPING ----------------
banner("TEST 6: Display sorted by category and grouping")
book.display(sort_by="category")
print()
for cat, names in sorted(book.group_by_category().items()):
    print(f"{cat:<10}: {', '.join(names)}")

# ---------------- Test 7: EMPTY ----------------
banner("TEST 7: Empty contact book")
ContactBook().display()

# ---------------- RWE: organisation scenario ----------------
banner("REAL-WORLD SCENARIO: Fest organising committee")
fest = ContactBook()
committee = [
    ("Prof. Lakshmi Iyer", "9840011223", "lakshmi.iyer@college.edu",  "faculty"),
    ("Dr. Anand Menon",    "9840099887", "anand.menon@college.edu",   "faculty"),
    ("TechNova Pvt Ltd",   "8041122334", "partners@technova.com",     "sponsor"),
    ("GreenLeaf Foods",    "8042233445", "sales@greenleaf.in",        "sponsor"),
    ("Sound Wave Events",  "9900112233", "bookings@soundwave.in",     "vendor"),
    ("Print Hub",          "9900445566", "orders@printhub.in",        "vendor"),
    ("Divya Sharma",       "7003344556", "divya.s@student.edu",       "volunteer"),
    ("Rahul Bose",         "7003344557", "rahul.b@student.edu",       "volunteer"),
    ("Imran Khan",         "7003344558", "imran.k@student.edu",       "volunteer"),
]
for c in committee:
    fest.add_contact(*c)
print("Step 1 - Full committee directory:")
fest.display()
print("\nStep 2 - Call list for all sponsors:")
fest.display(fest.search("sponsor", "category"))
print("\nStep 3 - Sound Wave Events changes its number; update by old phone:")
print("   old number was", fest.update_contact("9900112233", "phone", "9900998877"))
print("\nStep 4 - Imran Khan leaves the volunteer team; delete:")
print("   removed", fest.delete_contact("7003344558")["name"])
print("\nStep 5 - Contacts per category after changes:")
for cat, names in sorted(fest.group_by_category().items()):
    print(f"   {cat:<10}: {len(names)}  ({', '.join(names)})")
assert len(fest.contacts) == 8
assert fest.find_by_phone("9900998877")["name"] == "Sound Wave Events"

print("\nAll assertions passed.")
```

8.7 Test cases, output and verification (CT)

Each block is the genuine terminal output of the test program.

Test 1: Adding personal contacts

```
No.  Name               Phone        Category    Email
------------------------------------------------------------------------
1    Asha Rao           9876543210   Friend      asha.rao@gmail.com
2    Asha Verma         6001122334   Work        asha.v@yahoo.com
3    Dr. Suresh         7012345678   Work        suresh@college.edu
4    Karthik S          9000011111   Friend      -
5    Meera Nair         8899776655   Family      meera.nair@gmail.com
6    Ravi Kumar         9123456780   Family      ravi.k@outlook.com

Total: 6 contact(s)
```

Interpretation: Six contacts were stored. Names were normalised to Title Case ('asha rao' became 'Asha Rao') and the phone '98765 43210' was cleaned to 9876543210. The list is shown alphabetically, and Karthik, who has no email, shows '-'. Two contacts share the first name Asha, which is allowed because the phone number is the unique key.

Test 2: Searching

```
Search name 'asha' (partial, case-insensitive):
No.  Name               Phone        Category    Email
------------------------------------------------------------------------
1    Asha Rao           9876543210   Friend      asha.rao@gmail.com
2    Asha Verma         6001122334   Work        asha.v@yahoo.com

Total: 2 contact(s)

Search phone '9123':
No.  Name               Phone        Category    Email
------------------------------------------------------------------------
1    Ravi Kumar         9123456780   Family      ravi.k@outlook.com

Total: 1 contact(s)

Search category 'family':
No.  Name               Phone        Category    Email
------------------------------------------------------------------------
1    Meera Nair         8899776655   Family      meera.nair@gmail.com
2    Ravi Kumar         9123456780   Family      ravi.k@outlook.com

Total: 2 contact(s)

Search name 'zzz' (no match):
No contacts to display.
```

Interpretation: Searching 'ASHA' matched both Asha contacts (case-insensitive, partial). The phone search '9123' found Ravi, the category search returned both Family contacts, and a search with no match prints a friendly message instead of failing.

Test 3: Updating

```
Email changed from: (empty)
Category changed from: Friend
Phone changed from: 9123456780
No.  Name                Phone        Category    Email
----------------------------------------------------------------------
1    Karthik S           9000011111   Colleague   karthik.s@gmail.com

Total: 1 contact(s)
No.  Name                Phone        Category    Email
----------------------------------------------------------------------
1    Ravi Kumar          9123400000   Family      ravi.k@outlook.com

Total: 1 contact(s)
```

Interpretation: Karthik's email changed from empty to a real address, his category Friend became 'Colleague' (normalised), and Ravi's phone changed. The old phone now finds nobody while the new one finds Ravi, which proves the record was modified in place.

Test 4: Deleting

```
Deleted: Asha Verma | 6001122334
Contacts left: 5
Search 'verma' now -> no match (deletion verified)
Deleting again   -> caught: 'No contact found with phone 6001122334.'
```

Interpretation: Asha Verma was deleted, the count dropped from 6 to 5, and searching 'verma' returns nothing. Deleting the same person again raised a KeyError that the program caught.

Test 5: Validation and error handling

```
Empty name        : caught -> Name cannot be empty.
Short phone       : caught -> Invalid phone number '98765' (need 10 digits starting 6-9).
Starts with 5     : caught -> Invalid phone number '5876543210' (need 10 digits starting 6-9).
Letters in phone  : caught -> Invalid phone number '98ab543210' (need 10 digits starting 6-9).
Bad email         : caught -> Invalid email 'not-an-email'.
Duplicate phone   : caught -> A contact with phone 9876543210 already exists.
Update unknown    : caught -> 'No contact found with phone 9999999999.'
Bad update field  : caught -> Field must be one of ('name', 'phone', 'email', 'category').
Phone clash       : caught -> Phone 8899776655 already belongs to another contact.
Bad search field  : caught -> Search field must be one of ('name', 'phone', 'email', 'category').
```

Interpretation: Ten invalid operations were attempted: empty name, short phone, phone starting with 5, letters in phone, bad email, duplicate phone, update of an unknown contact, unknown field, phone clash and an invalid search field. All were rejected with clear messages and no data was corrupted.

Test 6: Display sorted by category and grouping

```
No.  Name                Phone        Category    Email
----------------------------------------------------------------------
1    Karthik S           9000011111   Colleague   karthik.s@gmail.com
2    Meera Nair          8899776655   Family      meera.nair@gmail.com
3    Ravi Kumar          9123400000   Family      ravi.k@outlook.com
4    Asha Rao            9876543210   Friend      asha.rao@gmail.com
5    Dr. Suresh          7012345678   Work        suresh@college.edu

Total: 5 contact(s)

Colleague : Karthik S
Family    : Ravi Kumar, Meera Nair
Friend    : Asha Rao
Work      : Dr. Suresh
```

Interpretation: Display can sort by any field. Sorting by category groups contacts, and group_by_category() builds a dictionary of lists using setdefault(). Family has two members.

Test 7: Empty contact book

```
No contacts to display.
```

Interpretation: An empty contact book prints 'No contacts to display.' with no crash.

8.8 Real-world application (RWE): organisational contact management

Scenario: a college fest organising committee must keep faculty coordinators, sponsors, vendors and student volunteers in one directory, pull a call list for one group, update a vendor's number and remove a volunteer who has left.

Real-World Scenario: Fest organising committee

```
Step 1 - Full committee directory:
No.  Name                Phone        Category    Email
----------------------------------------------------------------------
1    Divya Sharma        7003344556   Volunteer   divya.s@student.edu
2    Dr. Anand Menon     9840099887   Faculty     anand.menon@college.edu
3    Greenleaf Foods     8042233445   Sponsor     sales@greenleaf.in
4    Imran Khan          7003344558   Volunteer   imran.k@student.edu
5    Print Hub           9900445566   Vendor      orders@printhub.in
6    Prof. Lakshmi Iyer  9840011223   Faculty     lakshmi.iyer@college.edu
7    Rahul Bose          7003344557   Volunteer   rahul.b@student.edu
8    Sound Wave Events   9900112233   Vendor      bookings@soundwave.in
9    Technova Pvt Ltd    8041122334   Sponsor     partners@technova.com

Total: 9 contact(s)

Step 2 - Call list for all sponsors:
No.  Name                Phone        Category    Email
----------------------------------------------------------------------
1    Greenleaf Foods     8042233445   Sponsor     sales@greenleaf.in
2    Technova Pvt Ltd    8041122334   Sponsor     partners@technova.com

Total: 2 contact(s)

Step 3 - Sound Wave Events changes its number; update by old phone:
   old number was 9900112233

Step 4 - Imran Khan leaves the volunteer team; delete:
   removed Imran Khan

Step 5 - Contacts per category after changes:
   Faculty   : 2  (Prof. Lakshmi Iyer, Dr. Anand Menon)
   Sponsor   : 2  (Technova Pvt Ltd, Greenleaf Foods)
   Vendor    : 2  (Sound Wave Events, Print Hub)
   Volunteer : 2  (Divya Sharma, Rahul Bose)
```

Interpretation: The directory holds nine contacts in four groups. The sponsor call list was produced in one call by searching the category. The vendor's number was updated using the old phone as the key, the departed volunteer was deleted, and the final count shows two contacts per group (eight in total). Without a contact book, these changes would need manual edits in several places. Note that title() capitalises organisation names (TechNova appears as Technova); a production version could keep names as typed.

8.9 Conclusion

The contact book meets every requirement. A list of dictionaries gave flexible searching and sorting, validation prevented bad data, and 7 tests plus the fest scenario confirmed correct behaviour. The same design works for a personal address book or an organisation.

LAB EXERCISE 9	**Student grading system.** Student Grading System: Develop a Python program to store student names and marks using dictionaries, calculate grades using conditional statements, and display results.
CT	Apply conditional logic, analyze grade boundaries, and verify results using test cases. (CT)
RWE	Simulate an academic grading scenario and interpret student performance results. (RWE)

9.1 Aim

To store student names and subject marks in dictionaries, convert marks into grades with **if / elif / else**, display ranked results and statistics, and verify every grade boundary with test cases.

9.2 Requirement analysis (CT)

Requirement	Analysis	Implemented as
Store names and marks	Each student has marks in several subjects; the subject name should travel with the mark.	students = {name: {subject: marks}}
Calculate grades	Marks become a percentage, and the percentage maps to a grade band through conditional statements.	get_grade() with if / elif / else
Pass rule	A high average must not hide a failed subject, so failing any one subject fails the student.	result() checks failed_subjects()
Display results	Ranked table plus class statistics and grade distribution.	display_results(), display_statistics()
Validate input	Marks must be numbers from 0 to 100; every subject is required; no duplicate students.	add_student(), update_mark()
Verify	Check values just below, on and just above each boundary.	14 boundary tests, 7 tests, assertions

9.3 Grade boundaries (CT)

Percentage	Grade	Grade points	Remark	Condition used in code
90 and above	O	10	Outstanding	percentage >= 90
80 to below 90	A+	9	Excellent	percentage >= 80
70 to below 80	A	8	Very Good	percentage >= 70
60 to below 70	B+	7	Good	percentage >= 60
50 to below 60	B	6	Above Average	percentage >= 50
40 to below 50	C	5	Pass	percentage >= 40
Below 40	F	0	Fail	else

Why the order matters: the conditions run from the highest band downwards, so each one only needs a lower limit. A value of 89.99 fails >= *90* and falls into >= *80*, which is exactly right.

9.4 Selecting the data structure (CT)

Option	Verdict	Reason
Two lists (names, marks)	Rejected	Position-based; marks per subject need more lists and stay fragile.
List of tuples	Rejected	No direct lookup by name and subjects are not labelled.
Dictionary of dictionaries	**Chosen**	students['Aditya']['Maths'] is direct and self-explanatory; update_mark() changes one value; dictionary keys prevent duplicate students.
List of subjects	Used with it	Keeps the display order of subjects fixed.

9.5 Methods

Method	Purpose
get_grade(percentage)	if / elif / else chain returning (grade, points, remark); rejects values outside 0-100
add_student(name, marks) / update_mark(name, subject, mark)	Validated storage and correction of marks
total, percentage, failed_subjects	Calculations for one student
result(name)	Applies the pass rule: any subject below 40 gives overall grade F and status FAIL
ranking, subject_average, grade_distribution, class_statistics	Ranking, per-subject mean, count per grade, class summary
display_marks / display_results / display_statistics	Formatted output with a text bar chart of the grade distribution

9.6 Source code: grading_system.py

```
"""
Lab Exercise 9 : Student Grading System using Dictionaries
Author : Sriram Satvik Karanam | USN : 25BTRAD049 | CSE-AIDE B

Data structure used
-------------------
students : dict of dict   {student_name: {subject: marks}}
subjects : list           ordered subject names

Grade boundaries (percentage)         Grade  Points  Remark
    90 and above                        O      10    Outstanding
    80 to below 90                      A+      9    Excellent
    70 to below 80                      A       8    Very Good
    60 to below 70                      B+      7    Good
    50 to below 60                      B       6    Above Average
    40 to below 50                      C       5    Pass
    below 40                            F       0    Fail

Pass rule: a student must score at least 40 in EVERY subject. A student who
fails even one subject receives the overall grade F, whatever the average is.
"""

PASS_MARK = 40
MAX_MARKS = 100


def get_grade(percentage):
    """Convert a percentage into (grade, grade points, remark) using if/elif."""
    if percentage < 0 or percentage > 100:
```

```
        raise ValueError("Percentage must be between 0 and 100.")
    if percentage >= 90:
        return "O", 10, "Outstanding"
    elif percentage >= 80:
        return "A+", 9, "Excellent"
    elif percentage >= 70:
        return "A", 8, "Very Good"
    elif percentage >= 60:
        return "B+", 7, "Good"
    elif percentage >= 50:
        return "B", 6, "Above Average"
    elif percentage >= PASS_MARK:
        return "C", 5, "Pass"
    else:
        return "F", 0, "Fail"


class GradingSystem:
    def __init__(self, subjects):
        if not subjects:
            raise ValueError("At least one subject is required.")
        self.subjects = list(subjects)     # list
        self.students = {}                 # dict of dicts

    # ------------------------------------------------------------------
    # Store / update marks
    # ------------------------------------------------------------------
    def _check_mark(self, mark):
        if not isinstance(mark, (int, float)) or isinstance(mark, bool):
            raise ValueError("Marks must be a number.")
        if mark < 0 or mark > MAX_MARKS:
            raise ValueError(f"Marks must be between 0 and {MAX_MARKS}.")

    def add_student(self, name, marks):
        name = name.strip().title()
        if not name:
            raise ValueError("Student name cannot be empty.")
        if name in self.students:
            raise ValueError(f"Student '{name}' already exists.")
        if set(marks) != set(self.subjects):
            raise ValueError(f"Marks must be given for exactly: {', '.join(self.subjects)}.")
        for mark in marks.values():
            self._check_mark(mark)
        self.students[name] = dict(marks)

    def update_mark(self, name, subject, mark):
        name = name.strip().title()
        if name not in self.students:
            raise KeyError(f"Student '{name}' not found.")
        if subject not in self.subjects:
            raise KeyError(f"Subject '{subject}' not found.")
        self._check_mark(mark)
        old = self.students[name][subject]
        self.students[name][subject] = mark
        return old

    # ------------------------------------------------------------------
    # Calculations
    # ------------------------------------------------------------------
    def total(self, name):
        return sum(self.students[name].values())

    def percentage(self, name):
        return round(self.total(name) / (len(self.subjects) * MAX_MARKS) * 100, 2)

    def failed_subjects(self, name):
        return [s for s, m in self.students[name].items() if m < PASS_MARK]

    def result(self, name):
        """Return dict with total, percentage, grade, points, remark, status."""
        pct = self.percentage(name)
        failed = self.failed_subjects(name)
        if failed:
            grade, points, remark = "F", 0, "Fail"
            status = "FAIL"
        else:
```

```
            grade, points, remark = get_grade(pct)
            status = "PASS"
        return {"total": self.total(name), "percentage": pct, "grade": grade,
                "points": points, "remark": remark, "status": status, "failed": failed}

    def ranking(self):
        """Students sorted by percentage, highest first."""
        return sorted(self.students, key=lambda n: self.percentage(n), reverse=True)

    def subject_average(self, subject):
        return round(sum(m[subject] for m in self.students.values()) / len(self.students), 2)

    def grade_distribution(self):
        dist = {g: 0 for g in ("O", "A+", "A", "B+", "B", "C", "F")}
        for name in self.students:
            dist[self.result(name)["grade"]] += 1
        return dist

    def class_statistics(self):
        names = list(self.students)
        pcts = [self.percentage(n) for n in names]
        passed = sum(1 for n in names if self.result(n)["status"] == "PASS")
        return {"students": len(names),
                "class_average": round(sum(pcts) / len(pcts), 2),
                "highest": max(pcts), "lowest": min(pcts),
                "topper": self.ranking()[0],
                "pass_count": passed,
                "pass_percentage": round(passed / len(names) * 100, 2)}

    # ------------------------------------------------------------------
    # Display
    # ------------------------------------------------------------------
    def display_marks(self):
        short = [s[:7] for s in self.subjects]
        print(f"{'Student':<12}" + "".join(f"{s:>8}" for s in short))
        print("-" * (12 + 8 * len(short)))
        for name, marks in self.students.items():
            print(f"{name:<12}" + "".join(f"{marks[s]:>8}" for s in self.subjects))

    def display_results(self):
        if not self.students:
            print("No students recorded yet.")
            return
        print(f"{'Rank':<5}{'Student':<12}{'Total':>6}{'  %':>8}{'Grade':>7}{'GP':>4}  {'Result':<6} Remark")
        print("-" * 66)
        for rank, name in enumerate(self.ranking(), start=1):
            r = self.result(name)
            extra = f"  (failed: {', '.join(r['failed'])})" if r["failed"] else ""
            print(f"{rank:<5}{name:<12}{r['total']:>6}{r['percentage']:>8.2f}{r['grade']:>7}"
                  f"{r['points']:>4}  {r['status']:<6} {r['remark']}{extra}")

    def display_statistics(self):
        st = self.class_statistics()
        print(f"Students          : {st['students']}")
        print(f"Class average     : {st['class_average']}%")
        print(f"Highest / Lowest  : {st['highest']}% / {st['lowest']}%")
        print(f"Topper            : {st['topper']}")
        print(f"Pass percentage   : {st['pass_percentage']}% ({st['pass_count']} of {st['students']})")
        print("Subject averages  :", ", ".join(f"{s} {self.subject_average(s)}" for s in self.subjects))
        print("Grade distribution:")
        for g, n in self.grade_distribution().items():
            print(f"   {g:<3} {'#' * n:<10} {n}")


# ------------------------------------------------------------------
# Menu driven program
# ------------------------------------------------------------------
def menu():
    gs = GradingSystem(["Python", "Maths", "Physics", "English", "DS"])
    options = """
===== STUDENT GRADING SYSTEM =====
1. Add student marks     4. Show statistics
2. Update a mark         5. Exit
3. Show results
"""
    while True:
```

```
        print(options)
        choice = input("Enter choice: ").strip()
        try:
            if choice == "1":
                name = input("Student name: ")
                marks = {s: float(input(f"  {s} marks: ")) for s in gs.subjects}
                gs.add_student(name, marks)
                print("Student added.")
            elif choice == "2":
                old = gs.update_mark(input("Student: "), input("Subject: "), float(input("New marks: ")))
                print("Updated. Old marks were", old)
            elif choice == "3":
                gs.display_results()
            elif choice == "4":
                gs.display_statistics()
            elif choice == "5":
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")
        except (ValueError, KeyError) as err:
            print("Error:", err)


if __name__ == "__main__":
    menu()
```

9.7 Test program: test_grading.py

```python
from grading_system import GradingSystem, get_grade


def banner(title):
    print("=" * 66)
    print(title)
    print("=" * 66)


SUBJECTS = ["Python", "Maths", "Physics", "English", "DS"]

# ---------------- Test 1: grade boundaries ----------------
banner("TEST 1: Grade boundary analysis (get_grade)")
boundary_cases = [
    (100, "O"), (90, "O"), (89.99, "A+"), (80, "A+"), (79.99, "A"),
    (70, "A"), (69.99, "B+"), (60, "B+"), (59.99, "B"), (50, "B"),
    (49.99, "C"), (40, "C"), (39.99, "F"), (0, "F"),
]
print(f"{'Percentage':>11}  {'Expected':<9}{'Actual':<8}Verdict")
print("-" * 40)
all_ok = True
for pct, expected in boundary_cases:
    actual = get_grade(pct)[0]
    ok = actual == expected
    all_ok = all_ok and ok
    print(f"{pct:>11}  {expected:<9}{actual:<8}{'PASS' if ok else 'FAIL'}")
print("\nAll 14 boundary tests correct:", all_ok)
assert all_ok

# ---------------- Build the class ----------------
gs = GradingSystem(SUBJECTS)
class_data = {
    "Aditya":  [95, 92, 90, 88, 94],
    "Bhavana": [85, 82, 88, 79, 86],
    "Charan":  [72, 75, 68, 80, 71],
    "Deepa":   [65, 60, 70, 62, 68],
    "Eshwar":  [55, 58, 52, 60, 50],
    "Farah":   [45, 48, 42, 50, 44],
    "Gautam":  [30, 35, 28, 40, 32],
    "Harini":  [95, 95, 95, 95, 35],
    "Ishaan":  [100, 100, 100, 100, 100],
    "Jaya":    [40, 40, 40, 40, 40],
}
for name, marks in class_data.items():
    gs.add_student(name, dict(zip(SUBJECTS, marks)))

# ---------------- Test 2: stored marks ----------------
banner("TEST 2: Marks stored in the dictionary")
gs.display_marks()

# ---------------- Test 3: results ----------------
banner("TEST 3: Results ranked by percentage")
gs.display_results()
assert gs.result("Aditya")["grade"] == "O"
assert gs.result("Bhavana")["grade"] == "A+"
assert gs.result("Charan")["grade"] == "A"
assert gs.result("Deepa")["grade"] == "B+"
assert gs.result("Eshwar")["grade"] == "B"
assert gs.result("Farah")["grade"] == "C"
assert gs.result("Gautam")["grade"] == "F"
assert gs.result("Jaya")["grade"] == "C"

# ---------------- Test 4: failing one subject ----------------
banner("TEST 4: Fail-in-one-subject rule (Harini)")
r = gs.result("Harini")
print(f"Harini's percentage      : {r['percentage']}%")
print(f"Grade from percentage    : {get_grade(r['percentage'])[0]}")
print(f"Failed subjects          : {r['failed']}")
print(f"Final grade / result     : {r['grade']} / {r['status']}")
assert r["grade"] == "F" and r["status"] == "FAIL"

# ---------------- Test 5: update and regrade ----------------
banner("TEST 5: Correcting a mark and regrading")
print("Harini's DS marks were", gs.update_mark("harini", "DS", 45), "-> now 45")
```

```
r = gs.result("Harini")
print(f"New percentage           : {r['percentage']}%")
print(f"New grade / result       : {r['grade']} / {r['status']} ({r['remark']})")
assert r["grade"] == "A+" and r["status"] == "PASS"

# ---------------- Test 6: validation ----------------
banner("TEST 6: Validation and error handling")
good = dict(zip(SUBJECTS, [70, 70, 70, 70, 70]))
bad_cases = [
    ("Marks above 100",   lambda: gs.add_student("Kiran", {**good, "Maths": 101})),
    ("Negative marks",    lambda: gs.add_student("Kiran", {**good, "Maths": -5})),
    ("Text as marks",     lambda: gs.add_student("Kiran", {**good, "Maths": "ninety"})),
    ("Missing subject",   lambda: gs.add_student("Kiran", {"Python": 70})),
    ("Extra subject",     lambda: gs.add_student("Kiran", {**good, "Art": 80})),
    ("Duplicate student", lambda: gs.add_student("aditya", good)),
    ("Empty name",        lambda: gs.add_student("  ", good)),
    ("Unknown student",   lambda: gs.update_mark("Zed", "Python", 50)),
    ("Unknown subject",   lambda: gs.update_mark("Aditya", "Art", 50)),
    ("Bad percentage",    lambda: get_grade(101)),
]
for label, action in bad_cases:
    try:
        action()
        print(f"{label:<18}: NO ERROR (unexpected)")
    except (ValueError, KeyError) as e:
        print(f"{label:<18}: caught -> {e}")

# ---------------- Test 7: empty ----------------
banner("TEST 7: Empty system")
GradingSystem(SUBJECTS).display_results()

# ---------------- RWE: academic scenario ----------------
banner("REAL-WORLD SCENARIO: Semester results of a 10-student section")
gs2 = GradingSystem(SUBJECTS)
for name, marks in class_data.items():
    gs2.add_student(name, dict(zip(SUBJECTS, marks)))
gs2.display_results()
print()
gs2.display_statistics()
st = gs2.class_statistics()
weakest = min(SUBJECTS, key=gs2.subject_average)
strongest = max(SUBJECTS, key=gs2.subject_average)
print(f"\nStrongest subject : {strongest} (avg {gs2.subject_average(strongest)})")
print(f"Weakest subject   : {weakest} (avg {gs2.subject_average(weakest)})")
assert st["topper"] == "Ishaan"
assert st["pass_count"] == 8

print("\nAll assertions passed.")
```

9.8 Test cases, output and verification (CT)

Each block is the genuine terminal output of the test program.

Test 1: Grade boundary analysis (get_grade)

```
Percentage  Expected Actual  Verdict
---------------------------------------
       100  O        O       PASS
        90  O        O       PASS
     89.99  A+       A+      PASS
        80  A+       A+      PASS
     79.99  A        A       PASS
        70  A        A       PASS
     69.99  B+       B+      PASS
        60  B+       B+      PASS
     59.99  B        B       PASS
        50  B        B       PASS
     49.99  C        C       PASS
        40  C        C       PASS
     39.99  F        F       PASS
         0  F        F       PASS

All 14 boundary tests correct: True
```

Interpretation: All 14 values passed. The boundary values 90, 80, 70, 60, 50 and 40 land in the higher band, while 89.99, 79.99, 69.99, 59.99, 49.99 and 39.99 land in the band below. This confirms the >= comparisons are correct and no boundary is off by one.

Test 2: Marks stored in the dictionary

```
Student       Python   Maths Physics English      DS
-------------------------------------------------
Aditya            95      92      90      88      94
Bhavana           85      82      88      79      86
Charan            72      75      68      80      71
Deepa             65      60      70      62      68
Eshwar            55      58      52      60      50
Farah             45      48      42      50      44
Gautam            30      35      28      40      32
Harini            95      95      95      95      35
Ishaan           100     100     100     100     100
Jaya              40      40      40      40      40
```

Interpretation: Ten students are stored as a dictionary of dictionaries. Each mark is reached by student and subject name, which is easy to read and to update.

Test 3: Results ranked by percentage

```
Rank Student      Total       %  Grade  GP  Result Remark
-------------------------------------------------------------
1    Ishaan         500  100.00      O  10  PASS   Outstanding
2    Aditya         459   91.80      O  10  PASS   Outstanding
3    Bhavana        420   84.00     A+   9  PASS   Excellent
4    Harini         415   83.00      F   0  FAIL   Fail  (failed: DS)
5    Charan         366   73.20      A   8  PASS   Very Good
6    Deepa          325   65.00     B+   7  PASS   Good
7    Eshwar         275   55.00      B   6  PASS   Above Average
8    Farah          229   45.80      C   5  PASS   Pass
9    Jaya           200   40.00      C   5  PASS   Pass
10   Gautam         165   33.00      F   0  FAIL   Fail  (failed: Python, Maths, Physics, DS)
```

Interpretation: Ishaan (100%) and Aditya (91.8%) earn O. Jaya scored exactly 40 in every subject and passes with a C, which tests the pass mark itself. Gautam failed four subjects and gets F. Harini is ranked fourth with 83%, yet her grade is F, which is explained in Test 4.

Test 4: Fail-in-one-subject rule (Harini)

```
Harini's percentage      : 83.0%
Grade from percentage    : A+
Failed subjects          : ['DS']
Final grade / result     : F / FAIL
```

Interpretation: Harini's 83% would normally be A+, but she scored 35 in DS. Because the rule fails any student below 40 in any subject, her final grade is F with result FAIL. This shows why a grade from the average alone is not enough.

Test 5: Correcting a mark and regrading

```
Harini's DS marks were 35 -> now 45
New percentage           : 85.0%
New grade / result       : A+ / PASS (Excellent)
```

Interpretation: Correcting Harini's DS marks from 35 to 45 raised her percentage from 83% to 85%, and her result changed from F / FAIL to A+ / PASS. Grades are calculated on demand, so they always match the stored marks.

Test 6: Validation and error handling

```
Marks above 100   : caught -> Marks must be between 0 and 100.
Negative marks    : caught -> Marks must be between 0 and 100.
Text as marks     : caught -> Marks must be a number.
Missing subject   : caught -> Marks must be given for exactly: Python, Maths, Physics, English, DS.
Extra subject     : caught -> Marks must be given for exactly: Python, Maths, Physics, English, DS.
Duplicate student : caught -> Student 'Aditya' already exists.
Empty name        : caught -> Student name cannot be empty.
Unknown student   : caught -> "Student 'Zed' not found."
Unknown subject   : caught -> "Subject 'Art' not found."
Bad percentage    : caught -> Percentage must be between 0 and 100.
```

Interpretation: Ten invalid inputs were rejected: marks above 100, negative marks, text marks, missing subject, extra subject, duplicate student, empty name, unknown student, unknown subject and an invalid percentage.

Test 7: Empty system

```
No students recorded yet.
```

Interpretation: An empty system prints 'No students recorded yet.' instead of failing.

9.9 Real-world application (RWE): semester results of a section

Scenario: a faculty member enters the end-of-semester marks of ten students in five subjects, prints the ranked result sheet and uses the statistics to see how the section performed.

Real-World Scenario: Semester results of a 10-student section

```
Rank Student      Total       %  Grade  GP  Result Remark
-----------------------------------------------------------------
1    Ishaan         500  100.00      O  10  PASS   Outstanding
2    Aditya         459   91.80      O  10  PASS   Outstanding
3    Bhavana        420   84.00     A+   9  PASS   Excellent
4    Harini         415   83.00      F   0  FAIL   Fail  (failed: DS)
5    Charan         366   73.20      A   8  PASS   Very Good
6    Deepa          325   65.00     B+   7  PASS   Good
7    Eshwar         275   55.00      B   6  PASS   Above Average
8    Farah          229   45.80      C   5  PASS   Pass
9    Jaya           200   40.00      C   5  PASS   Pass
10   Gautam         165   33.00      F   0  FAIL   Fail  (failed: Python, Maths, Physics, DS)

Students          : 10
Class average     : 67.08%
Highest / Lowest  : 100.0% / 33.0%
Topper            : Ishaan
Pass percentage   : 80.0% (8 of 10)
Subject averages  : Python 68.2, Maths 68.5, Physics 67.3, English 69.4, DS 62.0
Grade distribution:
   O   ##         2
   A+  #          1
   A   #          1
   B+  #          1
   B   #          1
   C   ##         2
   F   ##         2

Strongest subject : English (avg 69.4)
Weakest subject   : DS (avg 62.0)
```

Interpretation: The class average is 67.08% and 8 of 10 students pass (80%). The marks range from 33% to 100%, so the section is mixed. The grade distribution has two O grades, one each of A+, A, B+ and B, two C and two F, so most of the section sits in the middle bands. **Data Structures (DS) has the lowest average at 62.0**, while English has the highest at 69.4. Harini's only failure is DS, so she needs targeted help in that subject, whereas Gautam is below 40 in four subjects and needs wider academic support. The weak DS average also suggests that the teaching or practice for that subject should be reviewed.

9.10 Conclusion

The grading system stores marks in dictionaries, assigns grades with conditional statements, and displays ranked results with statistics. Boundary testing proved the grade bands are exact, and the one-failed-subject rule prevented a misleading grade.

LAB EXERCISE 10	**Frequency counter using dictionaries.** Frequency Counter Using Dictionaries: Develop a Python program to count the frequency of words in a sentence or occurrences of elements in a list using dictionaries.
CT	Analyze input data, implement frequency counting, and verify the accuracy of the output. (CT)
RWE	Apply frequency counting to survey responses, classroom feedback, or product preferences and interpret the findings. (RWE)

10.1 Aim

To count how often each word appears in a sentence, or each element in a list, using a dictionary, to verify the counts against Python's built-in `collections.Counter`, and to interpret real survey, feedback and preference data.

10.2 Requirement analysis (CT)

Requirement	Analysis	Implemented as
Count words	'Python', 'python' and 'python.' must be the same word, so text is lower-cased and punctuation removed first.	clean_words(), word_frequency()
Count list elements	Elements may be numbers or text; optionally ignore case and stray spaces.	element_frequency(items, ignore_case)
Readable output	Sort by count (high to low), then alphabetically so ties are stable; show percentage and a bar.	sorted_frequency(), display_frequency()
Interpretation	Most and least common items with ties, top N, percentages, weighted average of ratings.	most_common(), top_n(), percentages()
Accuracy	Counts must add up to the number of items and match a trusted reference.	Comparison with Counter on 4 datasets
Robustness	Empty input, punctuation only, wrong input types.	Edge-case tests, TypeError

10.3 Selecting the data structure and algorithm (CT)

Option	Verdict	Reason
list.count(x) for every item	Rejected	Scans the whole list once per distinct item (slow for large data).
List of [item, count] pairs	Rejected	Finding an item needs a search every time.
Dictionary {item: count}	**Chosen**	One pass over the data; direct lookup by item; keys are unique by design.

Algorithm

- Clean the text (lower-case, strip punctuation) and split it into words; for lists use the elements directly.
- Create an empty dictionary. For every item execute `freq[item] = freq.get(item, 0) + 1`.
- Sort the pairs by `(-count, item)` and display count, percentage and a bar.

10.4 Functions

Function	Purpose
clean_words(text, remove_stop_words)	Lower-case, remove punctuation, split; optionally drop common words such as 'the' and 'is'
count_items(items)	The core counting loop using dict.get()
word_frequency / element_frequency	Counting for sentences and for lists (with optional ignore_case)
sorted_frequency, most_common, least_common, top_n	Ordering and ranking, with ties returned together
percentages, average_of_numeric	Share of each item; weighted mean for rating-type data
display_frequency(freq, title)	Table with count, percentage and text bar

10.5 Source code: frequency_counter.py

```
"""
Lab Exercise 10 : Frequency Counter using Dictionaries
Author : Sriram Satvik Karanam | USN : 25BTRAD049 | CSE-AIDE B

Core idea
---------
A dictionary maps each distinct item to the number of times it appears:
    freq = {item: count}
For every item we either start a new key at 1 or add 1 to the existing key:
    freq[item] = freq.get(item, 0) + 1
"""
import string

STOP_WORDS = {"a", "an", "the", "is", "are", "was", "were", "and", "or", "of", "to",
              "in", "on", "at", "it", "this", "that", "for", "with", "i", "we", "very"}


# ---------------------------------------------------------------------
# Counting
# ---------------------------------------------------------------------
def clean_words(text, remove_stop_words=False):
    """Lower-case the text, strip punctuation and split into a list of words."""
    if not isinstance(text, str):
        raise TypeError("Text must be a string.")
    table = str.maketrans("", "", string.punctuation)
    words = text.lower().translate(table).split()
    if remove_stop_words:
        words = [w for w in words if w not in STOP_WORDS]
    return words


def count_items(items):
    """Count how many times each element occurs in a list (or any iterable)."""
    freq = {}
    for item in items:
        freq[item] = freq.get(item, 0) + 1
    return freq


def word_frequency(text, remove_stop_words=False):
    return count_items(clean_words(text, remove_stop_words))


def element_frequency(items, ignore_case=False):
    if not isinstance(items, (list, tuple)):
        raise TypeError("Input must be a list or tuple.")
    if ignore_case:
        items = [i.strip().lower() if isinstance(i, str) else i for i in items]
    return count_items(items)


# ---------------------------------------------------------------------
# Analysis
```

```python
# ---------------------------------------------------------------------
def sorted_frequency(freq):
    """List of (item, count) sorted by count (high to low), then by item."""
    return sorted(freq.items(), key=lambda pair: (-pair[1], str(pair[0])))


def most_common(freq):
    """All items that share the highest count (handles ties)."""
    if not freq:
        return []
    top = max(freq.values())
    return sorted((k for k, v in freq.items() if v == top), key=str)


def least_common(freq):
    if not freq:
        return []
    low = min(freq.values())
    return sorted((k for k, v in freq.items() if v == low), key=str)


def top_n(freq, n):
    return sorted_frequency(freq)[:n]


def percentages(freq):
    total = sum(freq.values())
    return {k: round(v / total * 100, 1) for k, v in freq.items()} if total else {}


def average_of_numeric(freq):
    """Weighted mean when the keys are numbers (e.g. rating -> number of votes)."""
    total = sum(freq.values())
    return round(sum(k * v for k, v in freq.items()) / total, 2) if total else 0.0


# ---------------------------------------------------------------------
# Display
# ---------------------------------------------------------------------
def display_frequency(freq, title="Frequency table", bar_char="#"):
    if not freq:
        print("No data to display.")
        return
    total = sum(freq.values())
    width = max(len(str(k)) for k in freq) + 2
    print(title)
    print("-" * (width + 28))
    print(f"{'Item':<{width}}{'Count':>6}{'%':>8}  Bar")
    for item, count in sorted_frequency(freq):
        print(f"{str(item):<{width}}{count:>6}{count / total * 100:>7.1f}%  {bar_char * count}")
    print("-" * (width + 28))
    print(f"{'Total':<{width}}{total:>6}{'100.0%':>8}   ({len(freq)} distinct)")


# ---------------------------------------------------------------------
# Menu driven program
# ---------------------------------------------------------------------
def menu():
    options = """
===== FREQUENCY COUNTER =====
1. Count words in a sentence
2. Count elements in a list (comma separated)
3. Exit
"""
    while True:
        print(options)
        choice = input("Enter choice: ").strip()
        try:
            if choice == "1":
                text = input("Enter a sentence: ")
                skip = input("Ignore common words? (y/n): ").strip().lower() == "y"
                display_frequency(word_frequency(text, skip), "Word frequency")
            elif choice == "2":
                raw = input("Enter items separated by commas: ")
                items = [x.strip() for x in raw.split(",") if x.strip()]
                display_frequency(element_frequency(items, ignore_case=True), "Element frequency")
```

```
            elif choice == "3":
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")
        except TypeError as err:
            print("Error:", err)


if __name__ == "__main__":
    menu()
```

10.6 Test program: test_frequency.py

```
import random
from collections import Counter
from frequency_counter import (word_frequency, element_frequency, count_items, clean_words,
                               display_frequency, most_common, least_common, top_n,
                               percentages, average_of_numeric, sorted_frequency)


def banner(title):
    print("=" * 66)
    print(title)
    print("=" * 66)


# ---------------- Test 1: words in a sentence ----------------
banner("TEST 1: Word frequency in a sentence")
sentence = "the quick brown fox jumps over the lazy dog and the dog barks"
print("Sentence:", sentence)
freq = word_frequency(sentence)
display_frequency(freq, "Word frequency")
assert freq["the"] == 3 and freq["dog"] == 2 and freq["fox"] == 1
assert freq == dict(Counter(sentence.split()))

# ---------------- Test 2: case and punctuation ----------------
banner("TEST 2: Case and punctuation handling")
text = "Python is fun. PYTHON is powerful! Is python hard? No, python is easy."
print("Text:", text)
print("Cleaned words:", clean_words(text))
display_frequency(word_frequency(text), "Word frequency")
assert word_frequency(text)["python"] == 4
assert word_frequency(text)["is"] == 4

# ---------------- Test 3: elements in a list ----------------
banner("TEST 3: Element frequency in lists")
numbers = [4, 2, 7, 4, 4, 9, 2, 7, 4, 1]
print("Numbers:", numbers)
display_frequency(element_frequency(numbers), "Number frequency")
fruits = ["Apple", "banana", "apple", "Cherry", "BANANA", "apple ", "cherry"]
print("\nFruits (raw)   :", fruits)
print("Case-sensitive :", element_frequency(fruits))
print("Ignore case    :", element_frequency(fruits, ignore_case=True))
assert element_frequency(numbers)[4] == 4
assert element_frequency(fruits, ignore_case=True) == {"apple": 3, "banana": 2, "cherry": 2}

# ---------------- Test 4: stop words ----------------
banner("TEST 4: Removing common (stop) words")
line = "This is the best course and the course is very useful for the students"
print("Sentence:", line)
print("With stop words    :", sorted_frequency(word_frequency(line))[:4])
print("Without stop words :", sorted_frequency(word_frequency(line, True)))
assert "the" not in word_frequency(line, True)
assert word_frequency(line, True)["course"] == 2

# ---------------- Test 5: accuracy verification ----------------
banner("TEST 5: Accuracy verification against collections.Counter")
rng = random.Random(2026)
checks = 0
print(f"{'Dataset':<28}{'Items':>7}{'Distinct':>10}{'Sum of counts':>15}  Matches Counter")
print("-" * 76)
for label, size, pool in [("100 dice rolls", 100, list(range(1, 7))),
                          ("500 letters a-e", 500, list("abcde")),
                          ("1000 numbers 0-49", 1000, list(range(50))),
                          ("2000 colours", 2000, ["red", "green", "blue", "pink"])]:
    data = [rng.choice(pool) for _ in range(size)]
    mine = element_frequency(data)
    ok = mine == dict(Counter(data)) and sum(mine.values()) == size
    checks += ok
    print(f"{label:<28}{size:>7}{len(mine):>10}{sum(mine.values()):>15}  {ok}")
assert checks == 4

# ---------------- Test 6: edge cases ----------------
banner("TEST 6: Edge cases")
print("Empty sentence          ->", word_frequency(""))
print("Only punctuation        ->", word_frequency("?!... ,,"))
print("Single word             ->", word_frequency("hello"))
```

```
print("All same element        ->", element_frequency(["x"] * 5))
print("Mixed type keys         ->", element_frequency([1, "1", 1.0, "one"]))
tie = element_frequency(["a", "b", "b", "a", "c"])
print("Tie (a:2, b:2, c:1)     -> most common:", most_common(tie), "| least common:", least_common(tie))
for label, action in [("Number as text", lambda: word_frequency(12345)),
                      ("String as list", lambda: element_frequency("hello"))]:
    try:
        action()
    except TypeError as e:
        print(f"{label:<24}-> caught: {e}")
display_frequency({}, "Empty table")

# ---------------- RWE 1: survey ----------------
banner("REAL-WORLD 1: Survey - favourite programming language (30 students)")
responses = ["Python"] * 12 + ["Java"] * 7 + ["C++"] * 5 + ["JavaScript"] * 4 + ["Go"] + ["Rust"]
random.Random(7).shuffle(responses)
print("Responses:", ", ".join(responses[:10]), "... (30 in total)\n")
survey = element_frequency(responses)
display_frequency(survey, "Survey result")
print("\nMost popular   :", most_common(survey))
print("Least popular  :", least_common(survey))
print("Top 3          :", top_n(survey, 3))
share = percentages(survey)
print(f"Python + Java together: {share['Python'] + share['Java']:.1f}% of all votes")
assert sum(survey.values()) == 30 and survey["Python"] == 12

# ---------------- RWE 2: classroom feedback ratings ----------------
banner("REAL-WORLD 2: Classroom feedback - rating out of 5 (25 students)")
ratings = [5] * 9 + [4] * 8 + [3] * 5 + [2] * 2 + [1]
random.Random(11).shuffle(ratings)
fb = element_frequency(ratings)
display_frequency(dict(sorted(fb.items(), reverse=True)), "Rating distribution")
avg = average_of_numeric(fb)
satisfied = fb.get(4, 0) + fb.get(5, 0)
print(f"\nAverage rating        : {avg} / 5")
print(f"Satisfied (4 or 5)    : {satisfied} of {sum(fb.values())} = {satisfied / sum(fb.values()) * 100:.0f}%")
print(f"Dissatisfied (1 or 2) : {fb.get(1, 0) + fb.get(2, 0)} students")
assert avg == 3.88 and satisfied == 17

print("\nWritten comments (stop words removed), top 6 words:")
comments = [
    "The lab sessions were very helpful and practical.",
    "Practical lab sessions helped me understand Python.",
    "Need more time for lab sessions and more examples.",
    "Examples in the lectures were helpful.",
    "The pace of the lectures was too fast.",
    "More practical examples and slower pace please.",
]
words = word_frequency(" ".join(comments), remove_stop_words=True)
for word, count in top_n(words, 6):
    print(f"   {word:<12}{count}  {'#' * count}")

# ---------------- RWE 3: product preference ----------------
banner("REAL-WORLD 3: Product preference - cold drinks (24 customers)")
picks = (["Cola"] * 3 + ["cola"] * 2 + ["COLA "] * 2 + ["Lemon Soda"] * 5 + ["lemon soda"] * 2 +
         ["Orange"] * 5 + ["Mango"] * 3 + ["Water"] * 2)
random.Random(5).shuffle(picks)
prod = element_frequency(picks, ignore_case=True)
display_frequency(prod, "Product preference (case-insensitive)")
print("\nMost preferred :", most_common(prod), "<- tie")
print("Least preferred:", least_common(prod))
print("Without ignore_case the same data splits into", len(element_frequency(picks)),
      "'different' items instead of", len(prod))
assert most_common(prod) == ["cola", "lemon soda"]

print("\nAll assertions passed.")
```

10.7 Test cases, output and verification (CT)

Each block is the genuine terminal output of the test program.

Test 1: Word frequency in a sentence

```
Sentence: the quick brown fox jumps over the lazy dog and the dog barks
Word frequency
----------------------------------
Item    Count       %  Bar
the         3   23.1%  ###
dog         2   15.4%  ##
and         1    7.7%  #
barks       1    7.7%  #
brown       1    7.7%  #
fox         1    7.7%  #
jumps       1    7.7%  #
lazy        1    7.7%  #
over        1    7.7%  #
quick       1    7.7%  #
----------------------------------
Total      13  100.0%   (10 distinct)
```

Interpretation: The sentence has 13 words and 10 distinct words. 'the' appears 3 times (23.1%), 'dog' twice and every other word once. Words with equal counts are listed alphabetically, and the counts were checked against Counter.

Test 2: Case and punctuation handling

```
Text: Python is fun. PYTHON is powerful! Is python hard? No, python is easy.
Cleaned words: ['python', 'is', 'fun', 'python', 'is', 'powerful', 'is', 'python', 'hard', 'no', 'python', 'is', 'easy']
Word frequency
--------------------------------------
Item       Count       %  Bar
is             4   30.8%  ####
python         4   30.8%  ####
easy           1    7.7%  #
fun            1    7.7%  #
hard           1    7.7%  #
no             1    7.7%  #
powerful       1    7.7%  #
--------------------------------------
Total         13  100.0%   (7 distinct)
```

Interpretation: 'Python', 'PYTHON' and 'python' were counted as one word because the text is lower-cased and the punctuation is removed. 'python' and 'is' tie at 4 occurrences each, and the counts add up to the 13 cleaned words.

Test 3: Element frequency in lists

```
Numbers: [4, 2, 7, 4, 4, 9, 2, 7, 4, 1]
Number frequency
------------------------------
Item Count       %  Bar
4       4   40.0%  ####
2       2   20.0%  ##
7       2   20.0%  ##
1       1   10.0%  #
9       1   10.0%  #
------------------------------
Total    10  100.0%   (5 distinct)

Fruits (raw)   : ['Apple', 'banana', 'apple', 'Cherry', 'BANANA', 'apple ', 'cherry']
Case-sensitive : {'Apple': 1, 'banana': 1, 'apple': 1, 'Cherry': 1, 'BANANA': 1, 'apple ': 1, 'cherry': 1}
Ignore case    : {'apple': 3, 'banana': 2, 'cherry': 2}
```

Interpretation: The number 4 occurs four times. For the fruits, a case-sensitive count wrongly produces seven different items with a count of 1 each, whereas ignore_case merges them into apple 3, banana 2 and cherry 2 (the trailing space in 'apple ' is also removed).

Test 4: Removing common (stop) words

```
Sentence: This is the best course and the course is very useful for the students
With stop words    : [('the', 3), ('course', 2), ('is', 2), ('and', 1)]
Without stop words : [('course', 2), ('best', 1), ('students', 1), ('useful', 1)]
```

Interpretation: Common words such as 'the' (3) and 'is' (2) dominate the raw result. After stop-word removal, 'course' (2) becomes the leading meaningful word.

Test 5: Accuracy verification against collections.Counter

```
Dataset                      Items  Distinct  Sum of counts  Matches Counter
-------------------------------------------------------------------------
100 dice rolls                 100         6            100  True
500 letters a-e                500         5            500  True
1000 numbers 0-49             1000        50           1000  True
2000 colours                  2000         4           2000  True
```

Interpretation: Four datasets of 100 to 2000 items were counted and compared with Counter. The results were identical in every case, and the sum of the counts equalled the number of items, which is a useful correctness check for any frequency table.

Test 6: Edge cases

```
Empty sentence          -> {}
Only punctuation        -> {}
Single word             -> {'hello': 1}
All same element        -> {'x': 5}
Mixed type keys         -> {1: 2, '1': 1, 'one': 1}
Tie (a:2, b:2, c:1)     -> most common: ['a', 'b'] | least common: ['c']
Number as text          -> caught: Text must be a string.
String as list          -> caught: Input must be a list or tuple.
No data to display.
```

Interpretation: Empty input and punctuation-only input give an empty dictionary and a 'No data to display' message. A single word gives count 1. Python treats 1 and 1.0 as the same key, so [1, '1', 1.0, 'one'] gives {1: 2, '1': 1, 'one': 1}. A tie returns all top items, and wrong input types raise a clear TypeError.

10.8 Real-world application (RWE): survey, feedback and product preference

Three realistic data sets are analysed: a survey of favourite programming languages, classroom feedback (ratings and written comments) and a cold-drink preference poll.

Real-World 1: Survey - favourite programming language (30 students)

```
Responses: C++, Python, Java, Rust, Python, Go, Python, Python, Java, Python ... (30 in total)

Survey result
---------------------------------------
Item         Count       %  Bar
Python          12   40.0%  ############
Java             7   23.3%  #######
C++              5   16.7%  #####
JavaScript       4   13.3%  ####
Go               1    3.3%  #
Rust             1    3.3%  #
---------------------------------------
Total           30  100.0%   (6 distinct)

Most popular   : ['Python']
Least popular  : ['Go', 'Rust']
Top 3          : [('Python', 12), ('Java', 7), ('C++', 5)]
Python + Java together: 63.3% of all votes
```

Interpretation: Survey (30 students). Python is the clear favourite with 12 votes (40%), followed by Java (23.3%) and C++ (16.7%). Python and Java together hold 63.3% of the votes, while Go and Rust have a single vote each, so they are niche choices. A workshop or lab aimed at the section should therefore centre on Python.

Real-World 2: Classroom feedback - rating out of 5 (25 students)

```
Rating distribution
------------------------------
Item Count       %  Bar
5       9   36.0%  #########
4       8   32.0%  ########
3       5   20.0%  #####
2       2    8.0%  ##
1       1    4.0%  #
------------------------------
Total    25  100.0%   (5 distinct)

Average rating        : 3.88 / 5
Satisfied (4 or 5)    : 17 of 25 = 68%
Dissatisfied (1 or 2) : 3 students

Written comments (stop words removed), top 6 words:
   examples    3  ###
   lab         3  ###
   more        3  ###
   practical   3  ###
   sessions    3  ###
   helpful     2  ##
```

Interpretation: Classroom feedback (25 students). The average rating is 3.88 out of 5, and 17 of 25 students (68%) gave 4 or 5, while 3 students are dissatisfied. In the written comments, 'lab', 'sessions', 'practical', 'examples' and 'more' each appear three times, which shows that students value practical lab sessions and examples and want more of them. 'pace' and 'lectures' appear twice, pointing to a request for a slower lecture pace.

Real-World 3: Product preference - cold drinks (24 customers)

```
Product preference (case-insensitive)
--------------------------------------
Item         Count       %  Bar
cola             7   29.2%  #######
lemon soda       7   29.2%  #######
orange           5   20.8%  #####
mango            3   12.5%  ###
water            2    8.3%  ##
--------------------------------------
Total           24  100.0%   (5 distinct)

Most preferred : ['cola', 'lemon soda'] <- tie
Least preferred: ['water']
Without ignore_case the same data splits into 8 'different' items instead of 5
```

Interpretation: Product preference (24 customers). Cola and Lemon Soda tie at 7 votes each (29.2%), so neither is a single winner, and a shop should stock both equally. Orange follows with 5, Mango has 3 and Water is the least preferred with 2. Ignoring case merged eight written variants into five real products; without it the data would show 8 'different' items.

10.9 Conclusion

A dictionary counts items in a single pass, and the results matched Python's Counter on every dataset. Sorting by count and then alphabetically makes the output predictable, and percentages and averages turn raw counts into findings that can be acted on.

Summary and learning outcomes

All three lab exercises are complete, tested and interpreted.

Checklist

Requirement	Lab 8: Contact Book	Lab 9: Grading System	Lab 10: Frequency Counter
Uses lists / dictionaries	List of dictionaries	Dict of dicts + list of subjects	Dictionary {item: count}
Core operations	Add, search, update, delete, display	Store marks, grade, rank, display	Count words and list elements
CT: analysis and choice of structure	Section 8.2 and 8.3	Section 9.2, 9.3 and 9.4	Section 10.2 and 10.3
CT: verification	7 tests, 10 error cases	14 boundary tests, 7 tests	Counter comparison on 4 datasets
RWE scenario	Fest committee directory	Semester result of 10 students	Survey, feedback, product preference

What was learned

- A list of dictionaries suits records with several fields, and a unique key (the phone number) keeps updates and deletes safe.
- In an if / elif chain, order matters: testing from the highest band down keeps every boundary exact.
- Business rules (fail any subject means fail) can matter more than the raw average, so they must be tested explicitly.
- Boundary values, empty input and invalid input find bugs that normal data never shows.
- A dictionary turns counting into a single pass, and checking against collections.Counter is a simple way to verify accuracy.
- Raw counts only become useful when interpreted: shares, ties, averages and top-N lists lead to decisions.

How to run

```
python contact_book.py          # menu program, Lab 8
python grading_system.py        # menu program, Lab 9
python frequency_counter.py     # menu program, Lab 10
python test_contact_book.py     # tests and scenario, Lab 8
python test_grading.py          # tests and scenario, Lab 9
python test_frequency.py        # tests and scenarios, Lab 10
```